KANNADA SANGHA PUNE'S

KAVERI COLLEGE OF ARTS, SCIENCE & COMMERCE, PUNE

A PROJECT REPORT ON

DEALSCOUT

E-Commerce Product Comparison and Best-Deal Recommendation System

(HTML • CSS • JavaScript • Node.js)

T.Y.B.Sc. (Computer Science) — Semester V • Academic Year 2026–27

Submitted by

Name 1: _____ Seat No.: _____

Name 2: _____ Seat No.: _____

Name 3: _____ Seat No.: _____

Under the guidance of: _____

In partial fulfilment of B.Sc. (Computer Science), Savitribai Phule Pune University

KAVERI COLLEGE OF ARTS, SCIENCE & COMMERCE

PERMANENTLY AFFILIATED TO SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
Sr. No. 36, Shri G.M. Shetty Educational Complex, Ganeshnagar, Erandwane, Pune 411038.

CERTIFICATE

This is to certify that Mr. / Ms. _____, T.Y. B.Sc. (Computer Science) Semester V, has successfully completed his/her Project Work titled “DealScout” in Computer Science Specialization as a part of CS-331-FP Project as prescribed by Savitribai Phule Pune University for academic year 2026–27.

Seat No.: _____ Date: _____

Project Guide	**Course Coordinator**	**Principal**
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____

Examined by

Internal Examiner	**External Examiner**
Name: _____	Name: _____
Signature: _____	Signature: _____
Date: _____	Date: _____

INDEX

Sr. No.	Title	Page No.	Signature of Instructor
1	Declaration	5	
2	Acknowledgement	6	
3	Abstract	7	
4	1.1 Problem Identification	8	
5	1.2 Problem Statement / Definition	9	
6	1.3 Purpose, Objectives and Goals	10	
7	1.4 Technical Feasibility	11	
8	1.4 Economic, Operational and Schedule Feasibility	12	
9	1.5 Project Scope	13	
10	1.5 Limitations	14	
11	2.1 Hardware Requirements	15	
12	2.1 Software Requirements	16	
13	2.2 Frontend Technical Requirements	17	
14	2.2 Backend Technical Requirements (Node.js)	18	
15	2.2 Data Storage and Deployment Requirements	19	
16	2.3 Functional Requirements — Access	20	
17	2.3 Functional Requirements — Comparison	21	
18	2.4 Data Requirements	22	
19	2.4 Performance Requirements	23	
20	2.4 Security Requirements	24	
21	3.1 End Users	25	
22	3.2 Entities and Attributes	26	
23	3.2 ER Diagram	27	

INDEX (CONTINUED)

Sr. No.	Title	Page No.	Signature of Instructor
24	3.3 User File — users.json	28	
25	3.3 Product File — products.json and ProductOffer	29	
26	3.3 Search Log File — searches.json	30	
27	3.3 Relationships and Data Flow	31	
28	4.1 Class Diagram	32	
29	4.2 Object Diagram	33	
30	4.3 Component Diagram	34	
31	4.4 Deployment Diagram	35	
32	4.5 Use Case Diagram	36	
33	4.6 Activity Diagram	37	
34	4.7 Sequence Diagram	38	
35	4.8 System Architecture and Algorithm	39	
36	5.1.1 Login Screen	40	
37	5.1.2 Registration Screen (Proposed)	41	
38	5.1.3 Add Record Screen (Proposed)	42	
39	5.2.1 Dashboard	43	
40	5.2.2 Search Result Screen	44	
41	5.2.3 Insights Screen	45	
42	5.2.4 Resources and Quick Setup Screen	46	
43	Testing and Validation	47	
44	Conclusion and Future Scope	48	
45	Bibliography / References	49	
46	Appendix — Syllabus Mapping and Viva Checklist	50	

DECLARATION

I/We hereby declare that the work presented in this report entitled “DealScout” is an original study completed for T.Y. B.Sc. (Computer Science), Semester V, under the NEP 2020 pattern of Savitribai Phule Pune University.

- The system is implemented only with technologies taught in the Semester V curriculum: HTML, CSS, JavaScript (ES6+), Node.js and JSON file handling (CS-303-MJ-T Web Technology-I).
- The implementation and documentation are prepared for academic evaluation under CS-331-FP Project.
- All external references are acknowledged in the bibliography.
- Student names and seat numbers are intentionally left editable.

Placeholders should be completed and signed before final submission. The declaration confirms ownership of the submitted work while recognizing official documentation and software references.

ACKNOWLEDGEMENT

The completion of DealScout was made possible through guidance, review, testing, and access to academic resources.

- Gratitude to the Principal and Department of Computer Science.
- Thanks to the Project Guide and Course Coordinator.
- Acknowledgement of internal and external examiners.
- Appreciation for classmates, family, and the open-source Node.js and MDN communities.

Replace this text with personal acknowledgements if required by the assigned teacher. Keep the tone formal, concise, and connected to the actual support received.

ABSTRACT

DealScout is a web application for comparing offers for the same product across multiple sellers. It combines search, normalization, best-deal recommendation, login, and search-history insights in one responsive interface built with HTML, CSS, JavaScript and Node.js.

- Accepts product names and category filters from a browser form.
- Normalizes price, seller, rating, discount, and delivery fields into one JavaScript object structure.
- Highlights the lowest listed price as the best deal.
- Stores every search in a JSON file using the Node.js fs module.
- Supports a sample catalog and an optional live shopping provider.

The solution reduces repeated manual searching. Its design separates the browser interface (HTML/CSS/JS), the Node.js HTTP server, and JSON data files so that each part can be explained and maintained independently.

1.1 PROBLEM IDENTIFICATION

E-commerce platforms present similar products with different prices, discounts, ratings, seller names, and delivery estimates. Users must inspect each source separately.

- Manual comparison consumes time.
- Different field formats create confusion.
- A low visible price may exclude context such as seller or delivery.
- Search history is normally scattered or unavailable.
- Users need a transparent explanation of the recommended deal.

DealScout addresses these difficulties by displaying every normalized offer in one table and marking the minimum listed price. It does not claim that a preview price is live when no provider is connected.

1.2 PROBLEM STATEMENT / DEFINITION

To design and develop a responsive web system that searches product offers, converts them into a common structure, compares important values, identifies the lowest listed price, and stores search activity — using only HTML, CSS, JavaScript and Node.js.

- Input: product query, category, and source mode.
- Process: validation, retrieval, normalization, minimum-price selection, and logging.
- Output: best-deal card, comparison table, notices, and insights.
- Constraint: live results require a valid provider key; all other features work offline.

The problem definition is intentionally precise. It focuses on comparable offer data rather than promising purchase completion, payment handling, or guaranteed final checkout totals.

1.3 PURPOSE, OBJECTIVES AND GOALS

The purpose is to help users compare product offers with less effort and more context, while demonstrating the complete Web Technology-I skill set.

- Provide a clean and accessible search form (HTML5 forms and validation attributes).
- Support mobile, laptop, and general-product categories.
- Display seller, price, rating, delivery, and discount.
- Calculate the lowest price using understandable JavaScript logic.
- Protect insights behind a login session.
- Keep setup trivial: install Node.js and run one command.

Success is measured by whether a user can move from a query to a readable comparison and understand why one offer is highlighted. The interface must remain usable on desktop and mobile screens.

1.4 TECHNICAL FEASIBILITY

Every required technology is part of the Semester V syllabus, is free, and is well documented.

- HTML5 and CSS3 build the responsive interface (Unit: Frontend Fundamentals).
- JavaScript ES6+ (let/const, arrow functions, template literals, async/await, fetch, array methods) powers the browser logic (Unit: Advanced JavaScript).
- Node.js core modules http, path, url and crypto implement the server (Unit: Node.js Fundamentals).
- The fs module reads and writes JSON data files (Unit: File Handling with Node.js).
- An optional https call supplies live shopping results.

The sample catalog guarantees that core functionality works without any credentials or external service. No framework, database server, or third-party npm package is needed, making the architecture technically achievable for a student team.

1.4 ECONOMIC, OPERATIONAL AND SCHEDULE FEASIBILITY

Development can be completed with open-source tools and standard computer hardware, in achievable stages.

- Economic: Node.js, browsers, and editors such as VS Code are free; the sample catalog has no API cost.
- Economic: a live provider key is optional and only needed for real-time prices.
- Operational: clear labels, visible loading states, readable errors, and sign-in needed only for private insights.
- Schedule: requirements and data files first; server routes, pages, testing and documentation follow.
- Schedule: teacher feedback is applied before submission.

The modular sequence reduces project risk. Each stage produces a demonstrable output and supports independent correction without redesigning the entire system.

1.5 PROJECT SCOPE

DealScout covers product discovery and comparison for mobiles, laptops, and general items.

- Search a predefined sample catalog stored in products.json.
- Optionally retrieve live shopping results.
- Compare normalized offers.
- Highlight the minimum price.
- Store and summarize search activity in searches.json.
- Provide login, responsive design, downloadable resources, and quick setup scripts.

The scope ends before checkout. DealScout is an information and decision-support application, not a retailer, payment processor, inventory authority, or delivery service.

1.5 LIMITATIONS

Every comparison system depends on the quality, freshness, and completeness of its source data.

- Sample prices are not current retailer prices.
- Live mode requires a configured provider key.
- Taxes, exact variant, location, and shipping may affect final checkout cost.
- The demo login is not production identity management; passwords are stored in plain text in users.json for the preview only.
- JSON files suit a single-server educational workload, not thousands of concurrent writers.
- Retailer links may be unavailable in sample mode.

These limitations are shown honestly in the interface. Future work can add verified retailer partnerships, a database such as MongoDB (Semester V elective), hashed passwords, and variant matching.

2.1 HARDWARE REQUIREMENTS

DealScout is lightweight enough for an ordinary student computer or a college laboratory machine.

- Processor: dual-core 64-bit processor or better.
- Memory: minimum 2 GB RAM; 4 GB recommended.
- Storage: less than 50 MB for source code, data files and documents.
- Display: 1280×720 or higher recommended; the layout also adapts to phones.
- Network: optional — needed only for the live provider mode and for downloading Node.js once.

Because the server is a single Node.js process reading small JSON files, no dedicated database server or container runtime is required. The same machine runs both the server and the browser.

2.1 SOFTWARE REQUIREMENTS

Only software named in the Semester V practical course (CS-306-MJ-P) is required.

- Operating system: Windows 10/11, macOS, or Linux.
- Runtime: Node.js v14 or higher with npm (project runs without any npm package).
- Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.
- Editor: Visual Studio Code, Sublime Text, or Atom (optional for normal use).
- Documents: any PDF reader and Microsoft Word / LibreOffice for the report.

The quick setup scripts (setup.bat / setup.sh) check that Node.js exists, start server.js, and open http://localhost:3000 in the default browser.

2.2 FRONTEND TECHNICAL REQUIREMENTS

The frontend must be responsive, accessible, and independent of hard-coded server addresses.

- Semantic HTML5 elements: header, nav, main, section, footer, form, table.
- CSS3: custom properties, flexbox, grid, media queries, transitions and a keyframe animation.
- JavaScript ES6+: modules of functions, template literals, destructuring, spread, async/await.
- fetch() calls relative /api paths so the site works on any host or port.
- Every interactive element carries a data-testid attribute for browser testing.

The interface renders its shell even when server data is unavailable, and DOM updates are performed with innerHTML templates built from escaped values to avoid script injection.

```
const params = new URLSearchParams({ q: query, category, mode: state.mode });
state.result = await api(`/products/search?${params}`);
renderResults();
```

2.2 BACKEND TECHNICAL REQUIREMENTS (NODE.JS)

The server must serve static files, validate requests, and expose every route under /api as JSON — using only Node.js core modules.

- http.createServer() handles every request on one port.
- The url module (URL, URLSearchParams) parses paths and query strings.
- The path module safely maps URLs to files inside /public and blocks ../ traversal.
- The fs module streams static files and reads/writes JSON data.
- The crypto module generates unique ids and session tokens.

Routes are matched by a simple switch on “METHOD /path”, keeping the server easy to read for a viva. The provider key, when used, stays in a server environment variable and never reaches the browser.

```
const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  if (url.pathname.startsWith("/api")) await handleApi(req, res, url);
  else await serveStatic(req, res, url.pathname);
});
server.listen(process.env.PORT || 3000);
```

2.2 DATA STORAGE AND DEPLOYMENT REQUIREMENTS

Search activity must survive server restarts and remain separate from source code, without installing a database server.

- data/products.json — the sample catalog (read-only at runtime).
- data/users.json — demo login accounts.
- data/searches.json — append-only search log written with fs.promises.writeFile.
- Writes are queued one after another so concurrent searches never corrupt the file.
- Deployment = copy the folder and run node server.js; PORT and SERPAPI_KEY are optional environment variables.

Although the college guideline describes relational tables, the implementation uses equivalent JSON collections. The report identifies entities, fields, keys, and relationships clearly.

```
async function readJson(file, fallback = []) {
  try { return JSON.parse(await fs.promises.readFile(dataPath(file), "utf-8")); }
  catch (error) { if (error.code === "ENOENT") return fallback; throw error; }
}
```

2.3 FUNCTIONAL REQUIREMENTS — ACCESS

Access requirements define how visitors and signed-in users move through the application.

- FR-01: Visitor can open the comparison page.
- FR-02: Visitor can open the sign-in page.
- FR-03: Valid credentials create an HTTP-only session cookie.
- FR-04: Invalid credentials return HTTP 401 and a clear error.
- FR-05: Signed-in user can view shopping insights and sign out.
- FR-06: Guest cannot retrieve protected analytics data.

Authentication protects personal search summaries while leaving the core comparison experience publicly accessible.

2.3 FUNCTIONAL REQUIREMENTS — COMPARISON

The central functional requirements describe the complete search-to-recommendation flow.

- FR-07: Accept a query between 2 and 80 characters (HTML minlength/maxlength plus server check).
- FR-08: Filter by All, Mobiles, Laptops, or General.
- FR-09: Select Sample prices or Live search.
- FR-10: Display normalized offers in a table.
- FR-11: Identify the offer with minimum price.
- FR-12: Show a truthful notice when live data is unavailable.
- FR-13: Record every completed search in searches.json.

The highlighted result is based on listed price only. The UI retains seller, delivery, rating, and discount so the user can make a broader judgment.

2.4 DATA REQUIREMENTS

Data requirements define the minimum information needed to compare offers and summarize searches.

- User: id, name, email, password (preview only), and role.
- ProductOffer: id, product_name, category, source, seller, price, original_price, currency, rating, delivery, discount, product_url, mode, is_live, is_mocked, retrieved_at.
- SearchLog: id, query, category, mode, result_count, best_price, searched_at, sources.
- AnalyticsSummary: total_searches, unique_queries, average_results, top_query, source_counts, recent_searches.

String UUID values from crypto.randomUUID() are used as identifiers. Prices are JavaScript numbers and timestamps are ISO-8601 strings generated on the server.

2.4 PERFORMANCE REQUIREMENTS

The application should feel responsive for a small educational workload.

- Local sample searches return in well under one second.
- Live provider requests use an 8-second timeout.
- searches.json is capped at the most recent 500 records.
- Analytics reads at most 100 records.
- The UI shows “Finding offers…” while a request is pending.
- Static assets are sent with cache headers; HTML is not cached.

Performance depends on hardware and provider availability. The design avoids unbounded file growth and does not block the event loop with synchronous file access.

2.4 SECURITY REQUIREMENTS

Security requirements protect credentials, sessions, provider keys, and server files.

- Provider secret stays in a server environment variable.
- Sessions use HttpOnly, SameSite=Lax cookies that expire after 8 hours.
- Protected analytics validates the active session on every call.
- Query length, category and JSON body size are validated on the server.
- Static file paths are normalized and confined to the /public directory.
- All user-supplied text is HTML-escaped before rendering.

The bundled login is explicitly a preview account. Production use would add password hashing, HTTPS, and persistent sessions.

3.1 END USERS

Database design starts by identifying the people who use DealScout and the entities required by their actions.

- Visitor searches and compares offers without an account.
- Signed-in shopper reviews insights.
- Future administrator edits products.json and users.json.
- Evaluator or maintainer reviews behavior and stored records.

User roles are deliberately simple. Permissions are applied at the analytics endpoint, while comparison remains available without registration.

3.2 ENTITIES AND ATTRIBUTES

The data model contains User, ProductOffer, SearchLog, and AnalyticsSummary entities stored in or derived from JSON files.

- User primary key: id (users.json).
- ProductOffer primary key: id, generated when products.json is loaded.
- SearchLog primary key: id (searches.json).
- SearchLog.sources is an array of store names included in a comparison.
- AnalyticsSummary is calculated from SearchLog records rather than stored.

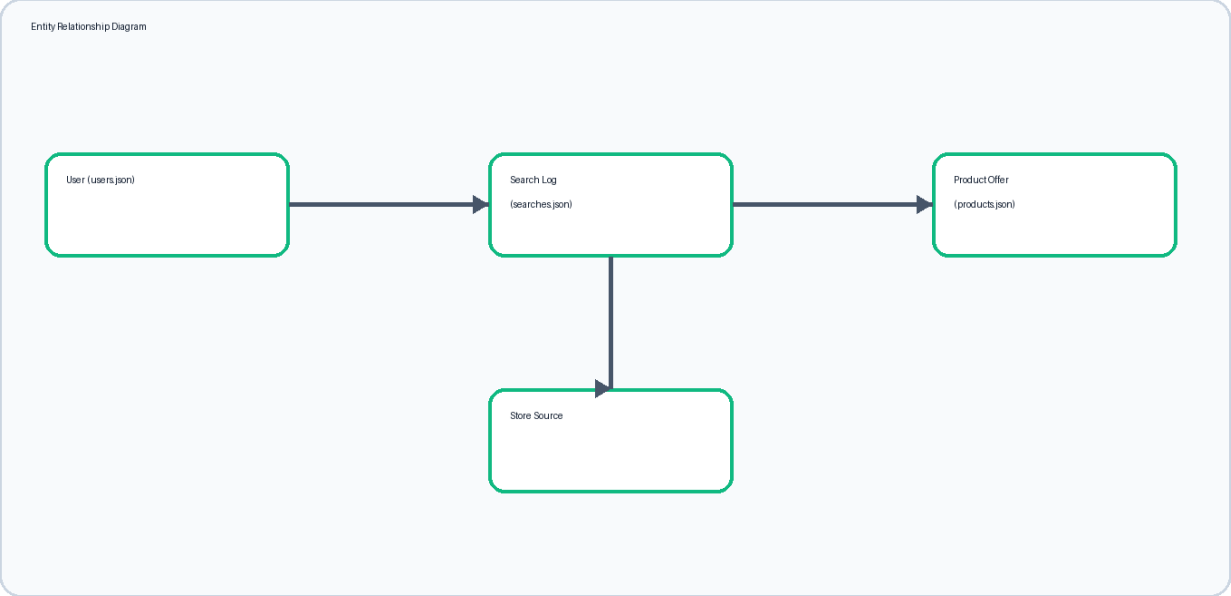
Entity attributes correspond directly to the JavaScript objects built in modules/products.js and consumed by public/js/app.js, so the same names appear in code, data files and this report.

3.2 ER DIAGRAM

The ER diagram shows the logical relationship between a shopper, searches, offers, and store sources.

- A user can review many search logs.
- A search can return many product offers.
- An offer belongs to one source at retrieval time.
- Analytics aggregates many search logs.

Each entity maps to one JSON file; adding an owner_id to searches.json is recommended for a true multi-user version.



3.3 USER FILE — users.json

The User record represents an authenticated identity used to open protected insights.

- id — string, primary identifier.
- name — string, display name.
- email — string, login identifier (compared case-insensitively).
- password — string, preview only.
- role — string, access role.

A production implementation would store a password hash and use a database with a unique email index.

```
[
  { "id": "preview-user-001", "name": "DealScout User",
    "email": "preview@dealscout.local", "role": "student" }
]
```

3.3 PRODUCT FILE — products.json AND ProductOffer

products.json holds the sample catalog; each row becomes a normalized ProductOffer object.

- price and original_price are numbers; discount is derived.
- currency defaults to INR.
- rating, delivery and product_url may be absent.
- mode identifies sample or live retrieval; is_live and is_mocked prevent misleading labels.
- retrieved_at communicates freshness.

Normalization allows a single comparison table and one minimum-price function to work across different sources.

```
{ "product_name": "Apple iPhone 15 128GB", "category": "Mobiles",
  "source": "Flipkart", "seller": "Flipkart Seller",
  "price": 68499, "original_price": 79900, "rating": 4.5, "delivery": "2 days" }
```

3.3 SEARCH LOG FILE — searches.json

searches.json records completed comparisons for later analysis; newest records are kept first.

- id — UUID string.
- query, category, mode — what the user asked for.
- result_count — number of normalized offers.
- best_price — minimum price or null.
- searched_at — ISO timestamp from the server.
- sources — store names returned by the search.

The file is rewritten atomically by a single queued writer, which is the JSON-file equivalent of a transaction.

```
{ "id": "6f1c…", "query": "iPhone 15", "category": "All", "mode": "demo",
  "result_count": 3, "best_price": 68499,
  "searched_at": "2026-10-06T19:25:11.000Z", "sources": ["Amazon", "Flipkart", "Croma"] }
```

3.3 RELATIONSHIPS AND DATA FLOW

The browser submits a query, the server produces ProductOffer objects, and one SearchLog is appended before the response returns.

- Logical User-to-SearchLog relationship: one-to-many.
- SearchLog-to-ProductOffer relationship: one-to-many during a response.
- Source-to-ProductOffer relationship: one-to-many.
- Analytics-to-SearchLog relationship: calculated aggregation using map, filter and reduce.

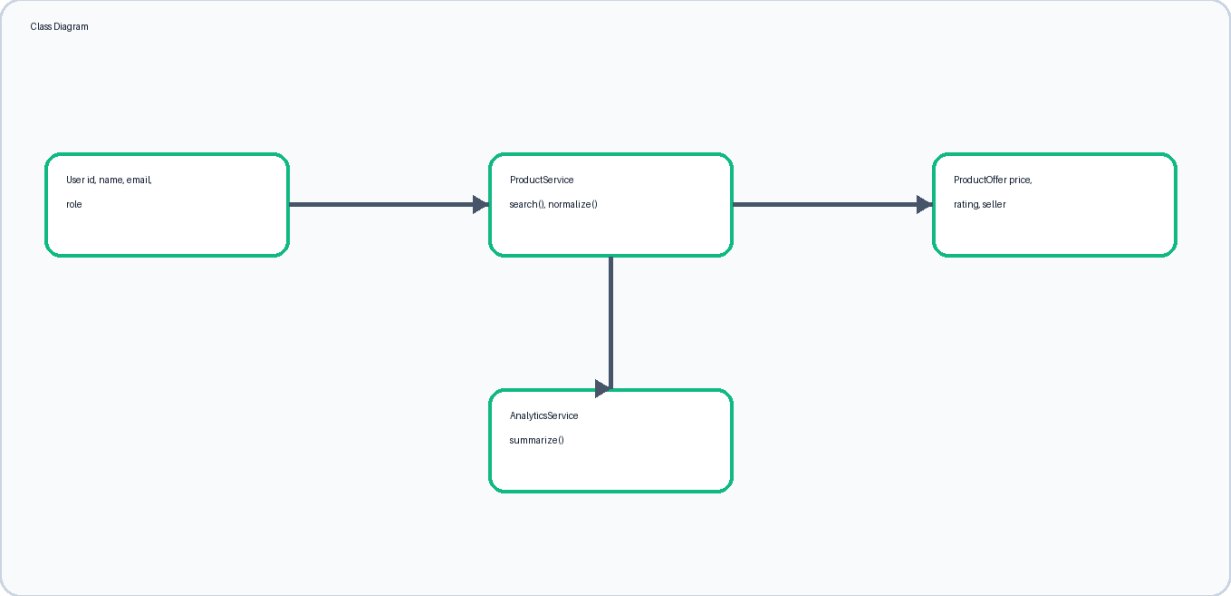
The application stores summary search data rather than full third-party payloads. This keeps the data files compact and limits unnecessary retention.

4.1 CLASS DIAGRAM

The class diagram connects data structures to the modules that use them.

- User contains identity and role data.
- ProductOffer contains comparable fields.
- ProductService (modules/products.js) retrieves, normalizes, and compares.
- AnalyticsService (modules/analytics.js) summarizes logs.
- Auth functions (modules/auth.js) create and validate sessions.

Although JavaScript modules are used instead of formal classes, each module has one responsibility, which is what the diagram shows.

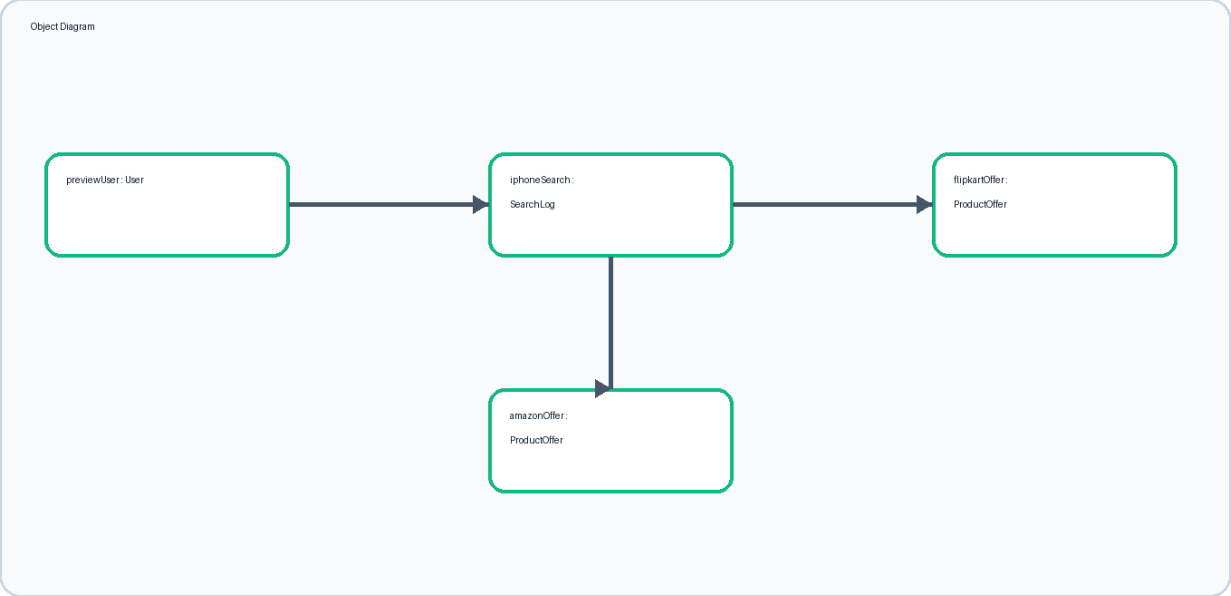


4.2 OBJECT DIAGRAM

The object diagram illustrates a single iPhone comparison at a moment in time.

- previewUser is an instance of User.
- iphoneSearch is an instance of SearchLog.
- flipkartOffer and amazonOffer are ProductOffer instances.
- The search object references returned source values.

Object diagrams help explain how data definitions become actual runtime values.

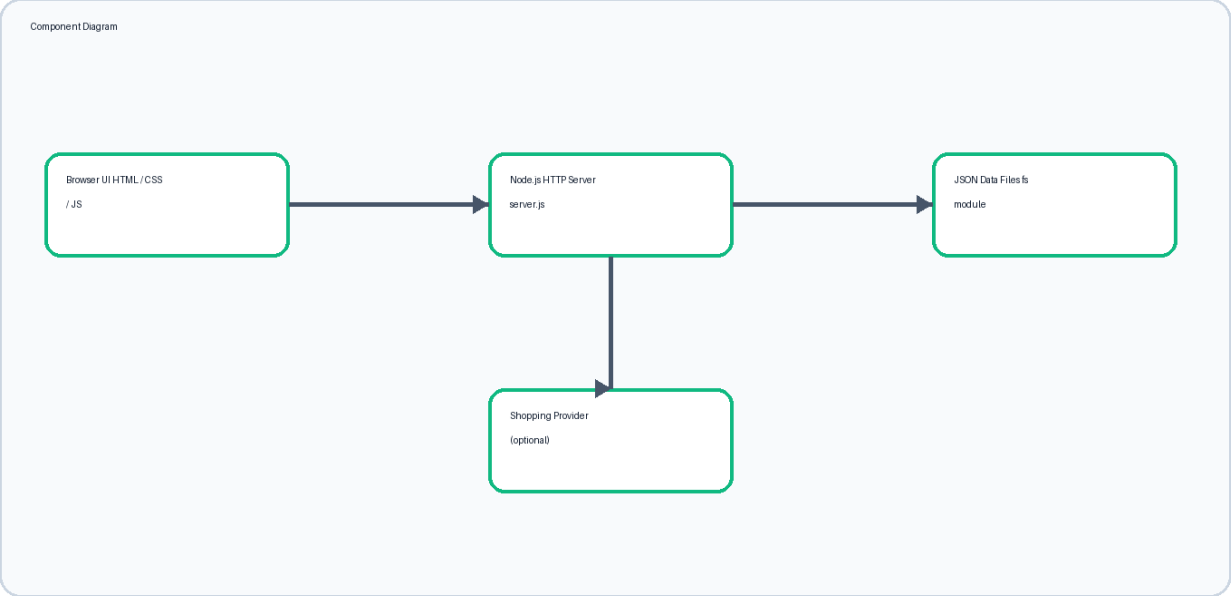


4.3 COMPONENT DIAGRAM

The component diagram displays the major software modules.

- Browser UI (HTML/CSS/JS) handles presentation and interaction.
- Node.js HTTP server validates and coordinates work.
- JSON data files persist catalog, users and search logs.
- Shopping provider optionally supplies live offers.

All browser-server communication uses JSON over relative /api routes.

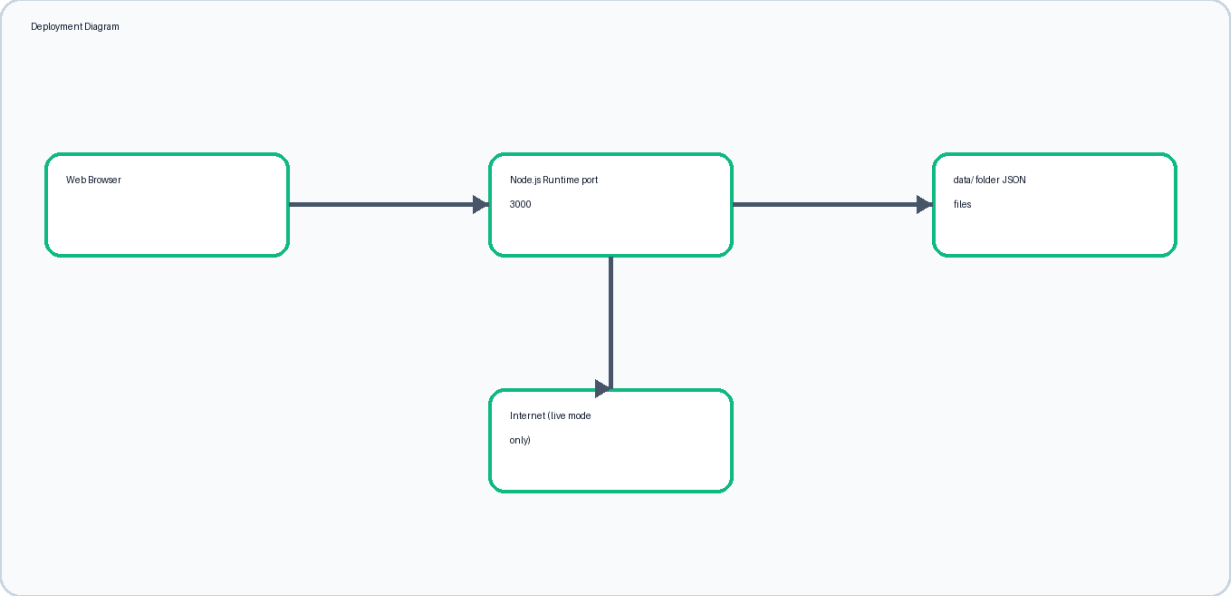


4.4 DEPLOYMENT DIAGRAM

A single Node.js process serves both the web pages and the API on one machine.

- The browser connects to localhost:3000 (or a chosen PORT).
- Node.js reads and writes the data/ folder.
- Internet access is used only in live mode.
- The same folder runs on Windows, macOS, or Linux.

No containers, web server software, or database service are needed.

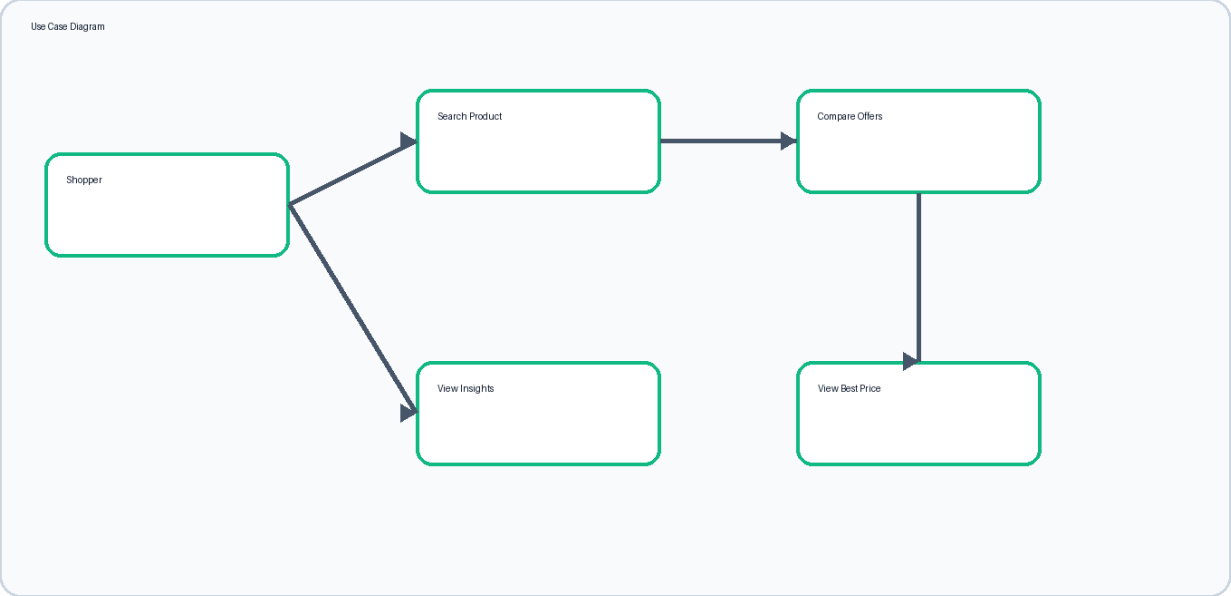


4.5 USE CASE DIAGRAM

Use cases describe behavior from the shopper's point of view.

- Search product.
- Choose category and data source.
- Compare normalized offers.
- View best price.
- Sign in / sign out.
- View comparison insights.
- Download support resources.

Administration and purchasing are outside the current scope.

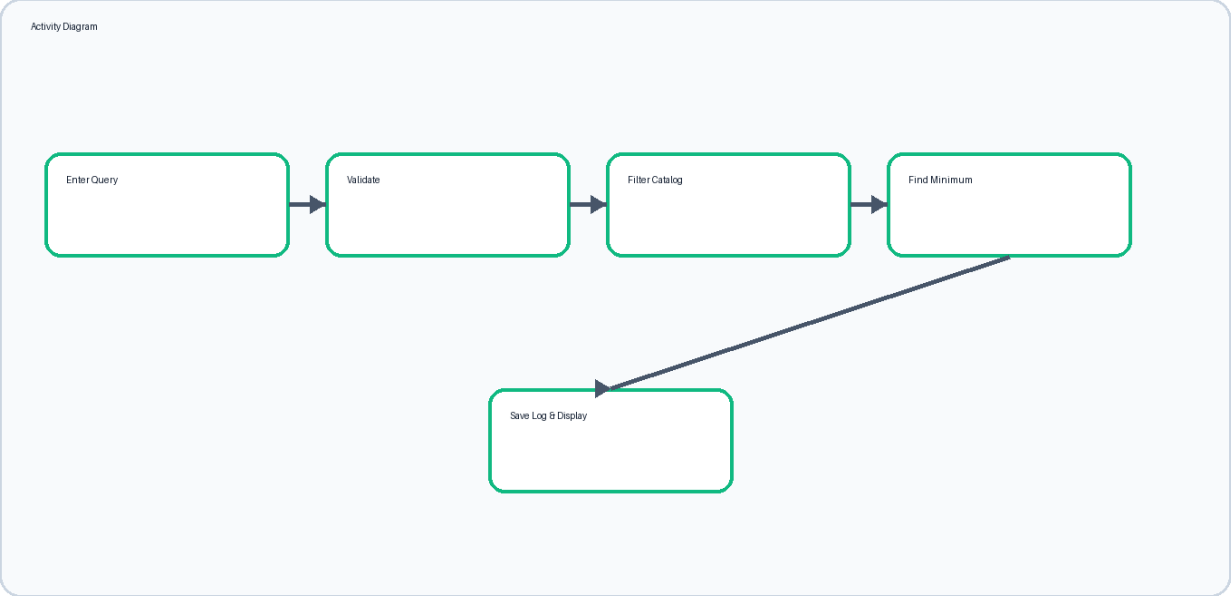


4.6 ACTIVITY DIAGRAM

The main activity begins with user input and ends with a displayed and recorded comparison.

- Enter and validate query.
- Filter the catalog or request the provider.
- Normalize accepted results.
- Find minimum price.
- Append a search log.
- Display notice, best-price card, and offer table.

If no offers match, the UI displays a useful empty state instead of a blank page.

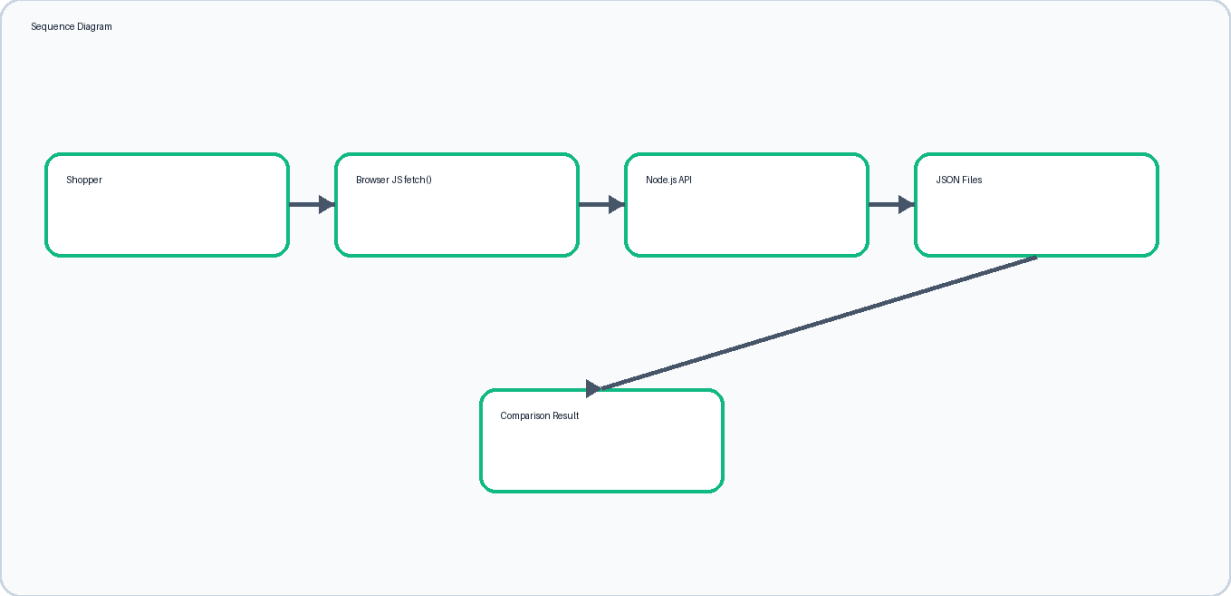


4.7 SEQUENCE DIAGRAM

The sequence diagram emphasizes message order between shopper, browser, server and files.

- Browser JavaScript sends GET /api/products/search via fetch().
- Server reads products.json (cached after first load).
- Server builds ProductOffer objects and selects the minimum.
- Server appends to searches.json.
- Server returns SearchResponse JSON.
- Browser renders cards and table.

Login follows a similar request sequence but creates an HTTP-only cookie.



4.8 SYSTEM ARCHITECTURE AND ALGORITHM

DealScout uses a layered client-server architecture and a simple minimum-price recommendation algorithm.

- Presentation layer: HTML pages styled with CSS, updated by JavaScript.
- API layer: Node.js http server with a route switch.
- Business layer: matching, normalization, and comparison.
- Data layer: JSON files accessed through the fs module.

Algorithm: collect matching offers; discard entries without a usable price; compare numeric prices with reduce(); return the id of the lowest as best_deal_id; the browser highlights that row and card.

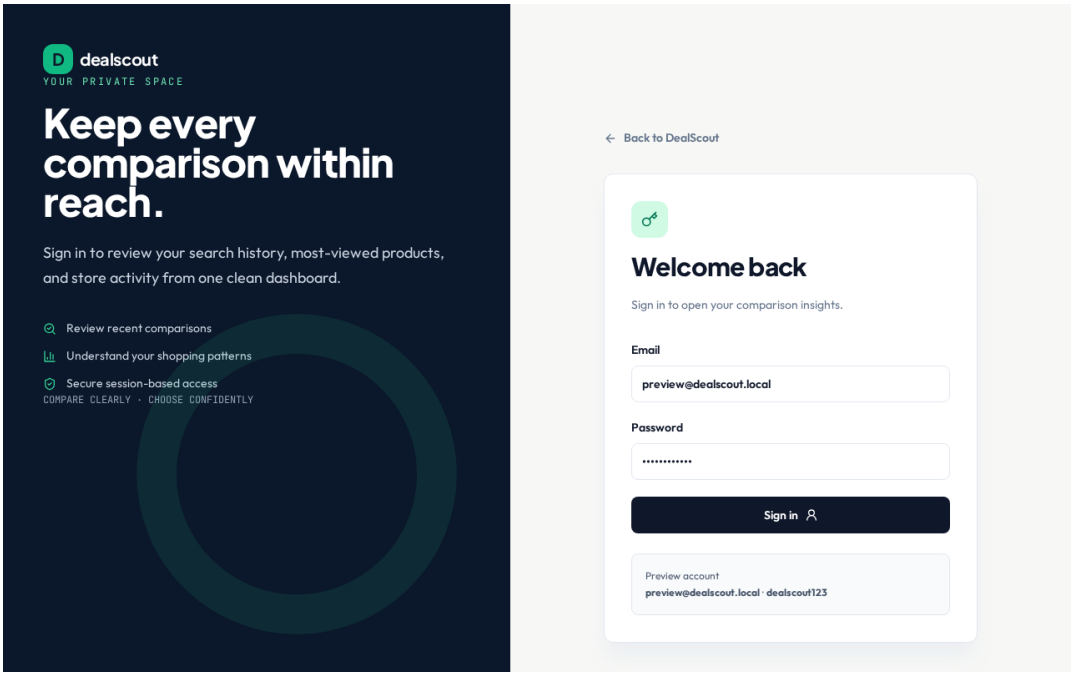
```
const best = results.reduce(
  (lowest, offer) => (lowest === null || offer.price < lowest.price ? offer : lowest),
  null,
);
```

5.1.1 LOGIN SCREEN

The login screen grants access to private shopping insights.

- Email input uses HTML5 type=email validation.
- Password input masks entered characters.
- Sign-in button submits credentials with fetch() POST.
- Invalid credentials display a concise error.
- Preview account details support evaluation.

The two-column layout separates the product message from the focused sign-in card.



5.1.2 REGISTRATION SCREEN (PROPOSED)

Registration is not implemented in the current version because only one evaluation account is required.

- Proposed fields: full name, email, password, confirm password.
- Proposed validation: unique email and password policy.
- Proposed output: new record in users.json and an active session.
- Passwords must be hashed before storage.

This documented future screen satisfies the planned multi-user extension without claiming that registration is available today.

5.1.3 ADD RECORD SCREEN (PROPOSED)

Catalog editing is currently done by editing products.json rather than through a public screen.

- Proposed administrator fields: product name, category, store, seller, price, original price, rating, delivery, and URL.
- Required validation: positive price, recognized category, safe URL.
- Actions: save, clear, cancel, and preview.

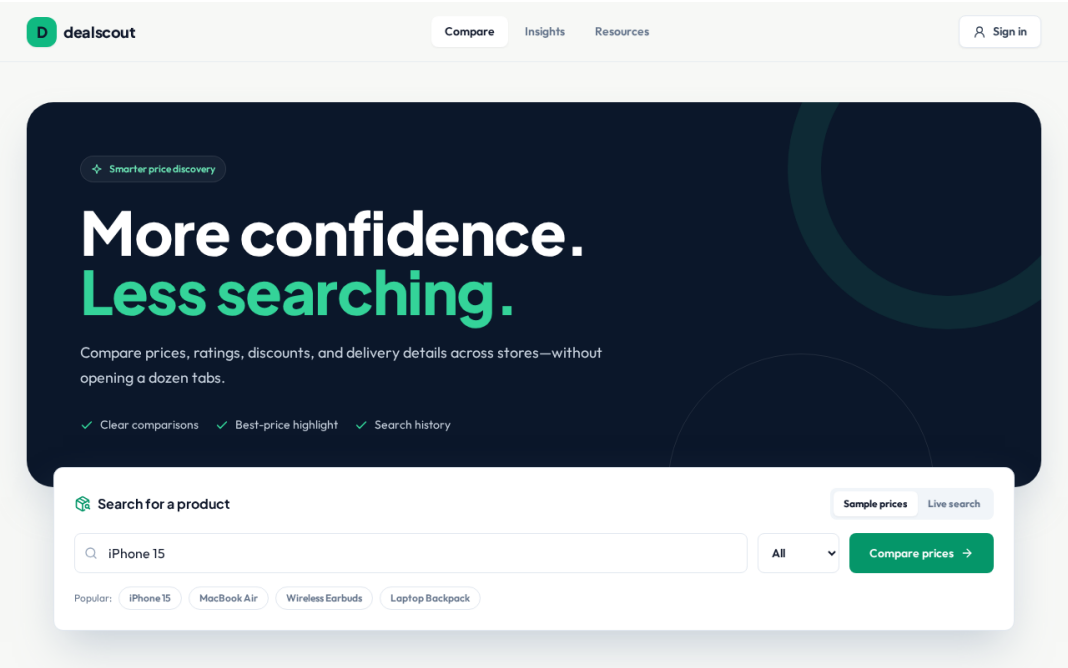
An authenticated administrator role should be introduced before this screen is enabled.

5.2.1 DASHBOARD

The dashboard gives immediate access to navigation, mode selection, category filter, product query, examples, and feature explanation.

- Hero communicates the service benefit.
- Search form is visually dominant.
- Sample and Live controls are clearly separated.
- Popular queries accelerate first use.

The page uses professional, consumer-facing language.

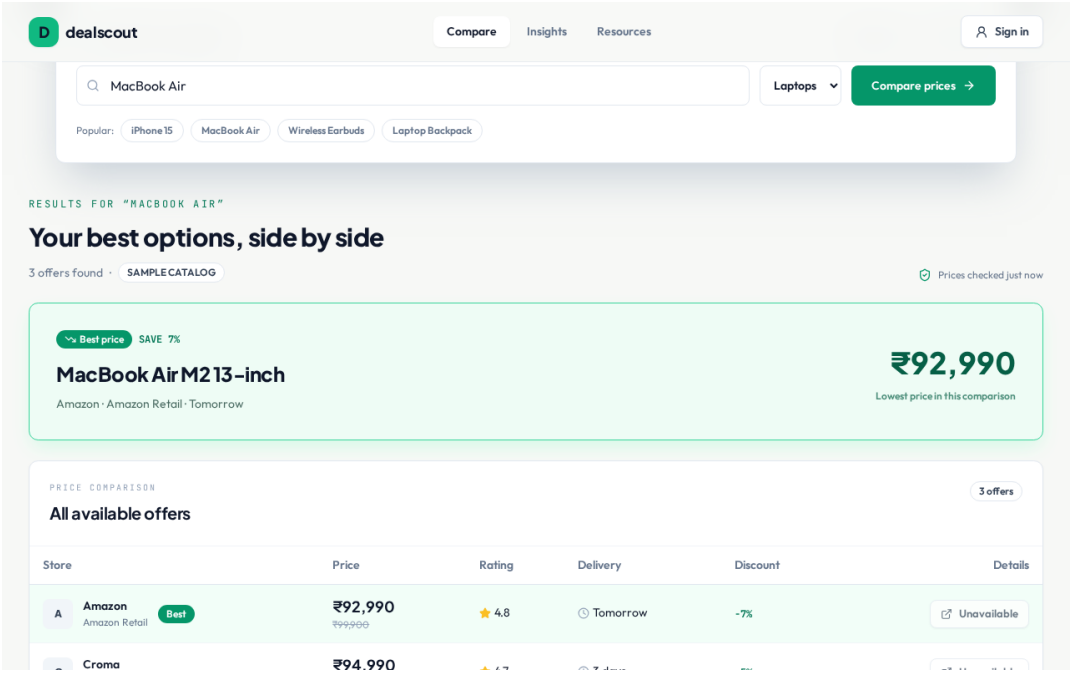


5.2.2 SEARCH RESULT SCREEN

Search results place the best price before the detailed comparison table.

- Query, result count, and source label remain visible.
- Preview-mode notice prevents misleading live claims.
- Best-price card shows product, store, seller, delivery, price, and discount.
- Table scrolls horizontally on narrow screens.

Emerald is reserved for savings and the best-price recommendation.

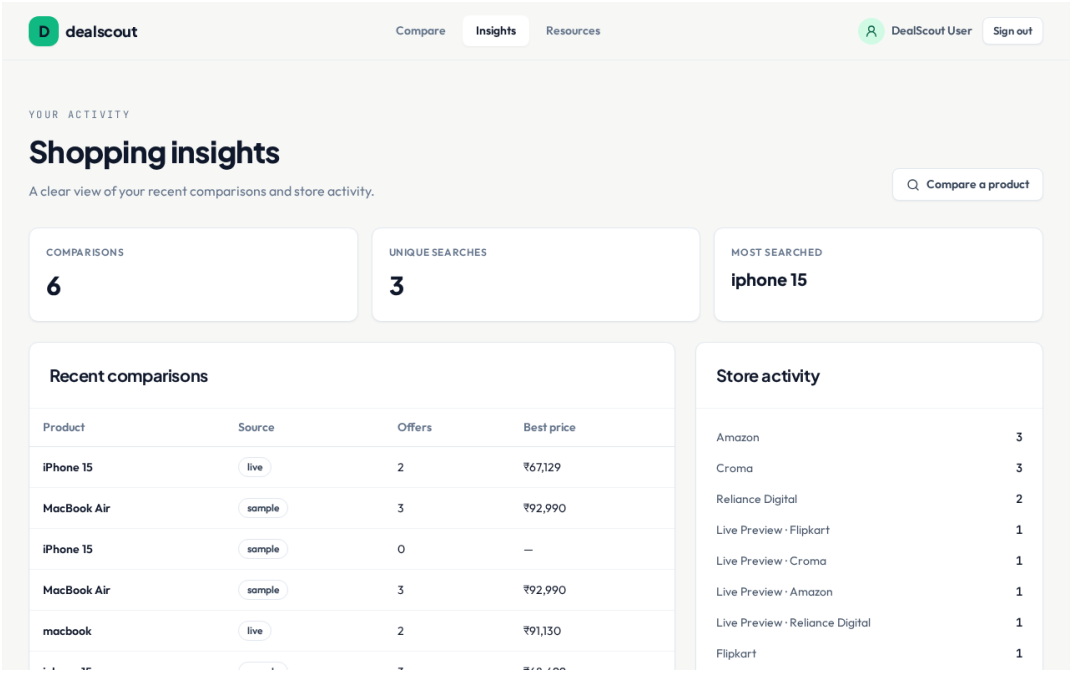


5.2.3 INSIGHTS SCREEN

The insights screen summarizes stored comparison activity for a signed-in user.

- Comparison count, unique search count and most searched query.
- Recent comparison table.
- Store frequency list.
- Sign-in prompt for guests.

Compact cards and tables keep the values easy to verify against searches.json.

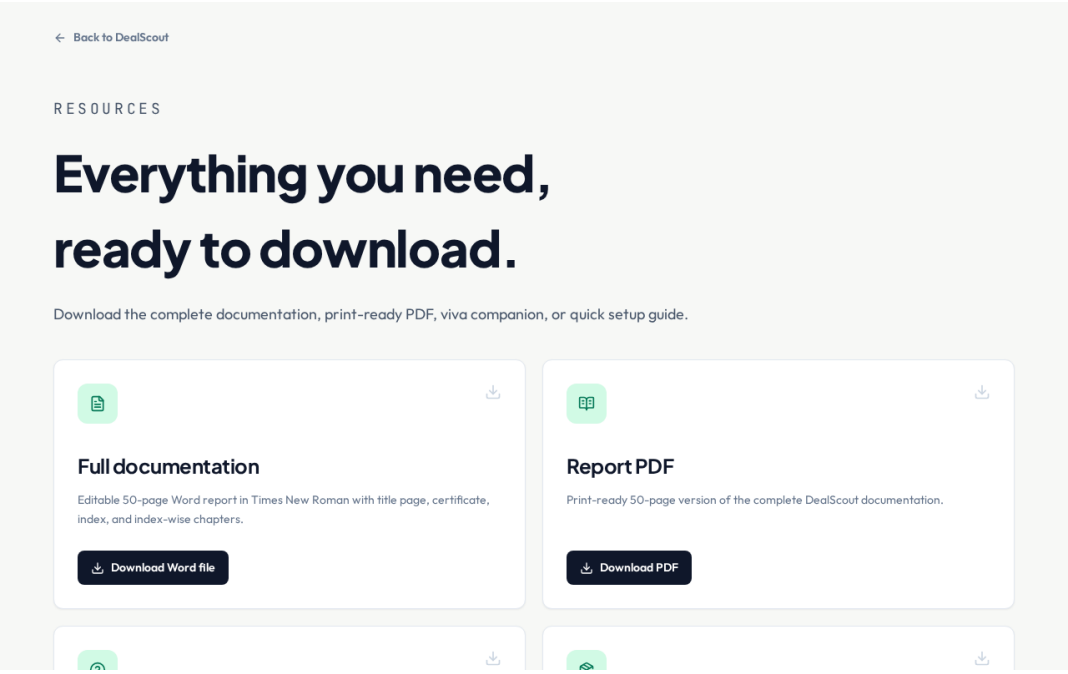


5.2.4 RESOURCES AND QUICK SETUP SCREEN

The resources page offers the documentation downloads and the quick setup instructions.

- Word report, PDF report, viva guide, and setup guide downloads.
- Step list: install Node.js, open the folder, run setup.bat or setup.sh.
- Single-command alternative: node server.js.

Files are served by the same Node.js static handler as every other asset.



TESTING AND VALIDATION

Testing confirms the main flow and important failure states across the API and the browser.

- GET /api/ returns ok: true.
- Valid search returns results and best_deal_id; the best offer has the lowest price.
- One-character query returns HTTP 400.
- Valid login returns the user and sets the session cookie; wrong password returns 401.
- GET /api/analytics returns 401 for guests and a summary after login.
- Unknown page returns the custom 404 page.
- Browser flow: search → best price → live preview notice → login → insights with no console errors.

A change is considered complete only when the exact failed check passes after correction.

CONCLUSION AND FUTURE SCOPE

DealScout demonstrates a complete comparison workflow using only the Semester V web stack: a responsive HTML/CSS interface, ES6+ browser logic, a Node.js server, and JSON file storage.

- Connect verified retailer or shopping-provider credentials.
- Add product-variant matching and currency normalization.
- Include shipping and tax in total-cost comparison.
- Move data to MongoDB (Semester V elective CS-309) and hash passwords.
- Add price alerts, favorites, and historical trends.
- Create an administrator catalog editor.

The current solution meets its defined objective while clearly separating implemented behavior from future enhancements.

BIBLIOGRAPHY / REFERENCES

The following sources support implementation and documentation.

- MDN Web Docs — HTML, CSS and JavaScript reference — https://developer.mozilla.org/
- Node.js Documentation — http, fs, path, url, crypto modules — https://nodejs.org/docs/
- ECMAScript 2015+ Language Specification — https://tc39.es/ecma262/
- SerpApi Google Shopping API — https://serpapi.com/google-shopping-api
- Savitribai Phule Pune University, T.Y.B.Sc. (Computer Science) NEP syllabus, CS-303-MJ-T Web Technology-I and CS-331-FP Project.
- Kaveri College project documentation instructions supplied for academic year 2026–27.

All product and retailer names are used for demonstration. Live data use must follow the chosen provider's terms, rate limits, attribution, and caching rules.

APPENDIX — SYLLABUS MAPPING AND VIVA CHECKLIST

This final page maps the implementation to the CS-303-MJ-T units and lists what to prepare before review.

- Unit 1 Frontend Fundamentals → index.html, login.html, resources.html, style.css.
- Unit 2 Advanced JavaScript (ES6+) → app.js, login.js, common.js (fetch, async/await, template literals, array methods).
- Unit 3 Node.js Fundamentals → server.js, modules/api.js (http, url, path, crypto, modules, npm scripts).
- Unit 4 File Handling with Node.js → modules/filestore.js and the data/*.json files.
- Fill names, seat numbers, guide, coordinator, principal, examiner, date and signatures.
- Carry the Word and PDF report, viva guide, source folder and setup guide.

Core viva explanation: DealScout receives a query, normalizes offers to one object structure, finds the minimum numeric price with reduce(), appends a search log to a JSON file, and displays both the recommendation and the supporting comparison rows.